Optimization Algorithms (Solvers)

Optimizers are the algorithms that determine how the weights of the machine learning model are tuned during backpropagation.

It is essential for deep learning.
training a complex deep learning model can take hours, days, or even weeks.

- the performance of the optimization algorithm direclty affects the model's training efficiency.
  They answer:
  "How do we *minimize* the loss function?"
- **Optimization Algorithm** is a tool that allowed us to continue updating model parameters and to minimize the value of the loss function, as evaluated on the training set.
- reduce the training error
- However, the goal of deep learning (or more broadly, statistical inference) is to reduce the generalization error.
- They do **NOT** define how wrong the model is, they define **how to update parameters** to reduce that wrongness.

There is no fixed number of optimization algorithms in machine learning; instead, they can be grouped into families such as gradient-based, second-order, quasi-Newton, stochastic variance-reduced, and heuristic methods.

What YOU actually need to remember (important)

TASK	OPTIMIZERS TO KNOW
Logistic / Linear Regression	Gradient Descent, Newton, **LBFGS**
Sparse models (L1)	Coordinate Descent, **SAGA**
Deep Learning	SGD, Adam, AdamW
Black-box problems	Genetic Algorithms

1. Newton-style methods (second-order methods)

What they are

Newton methods use:

- **First derivative (gradient)** → direction
- **Second derivative (Hessian)** → curvature
  So they know:

> "Which way is downhill *and* how steep the valley is."

Update rule (conceptual)

[
\beta*{new} = *beta*{old} - H^{-1} \nabla J(\beta)
]

Where:

- (\nabla J) = gradient
- (H) = Hessian matrix (second derivatives)

Why they're powerful

- Very **fast convergence**
- Often reach optimum in **few iterations**
- Excellent for **convex problems** like logistic regression

Why they're NOT always used

- Hessian is **expensive to compute**
- Inverting a matrix is slow
- Memory heavy for many features

Best for:

- Small to medium datasets
- Low number of features

2. LBFGS (Limited-memory BFGS)

What it is

LBFGS is a **smart approximation** of Newton's method.
Instead of computing the full Hessian:

- It **estimates curvature** using past gradients
- Stores only a **small amount of history**

Hence the name:

> **Limited-memory BFGS**

Why LBFGS is popular

- Much faster than vanilla gradient descent

- Uses second-order information (approximately)
- Memory efficient
- Stable for logistic regression

When LBFGS is best

Small–medium datasets
Dense features
L2 regularization
Binary or multiclass logistic regression

it does **not** support L1 penalty

3. SAGA (stochastic average gradient)

What it is

SAGA is a **stochastic gradient descent (SGD) variant**, but much smarter.
Instead of:

- Using full dataset each step (slow)
- Using random noisy steps (unstable)

SAGA:

- Stores **past gradients**
- Uses an **average gradient** for stability

Why SAGA is special

- Supports **L1 regularization** (feature selection)
- Handles **very large datasets**
- Scales well with many samples

Trade-off

- Slower per iteration than SGD
- Needs more iterations than LBFGS
- But **excellent for large-scale problems**

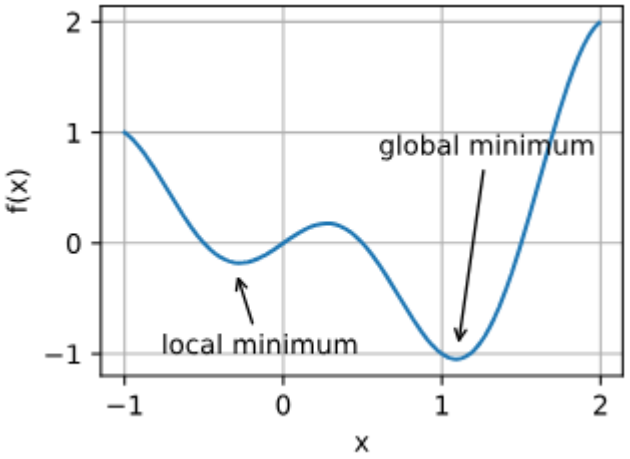
Side-by-side comparison

SOLVER	USES GRADIENT	USES CURVATURE	L1 SUPPORT	BEST FOR
Gradient Descent	✅	❌	❌	Learning / small tasks

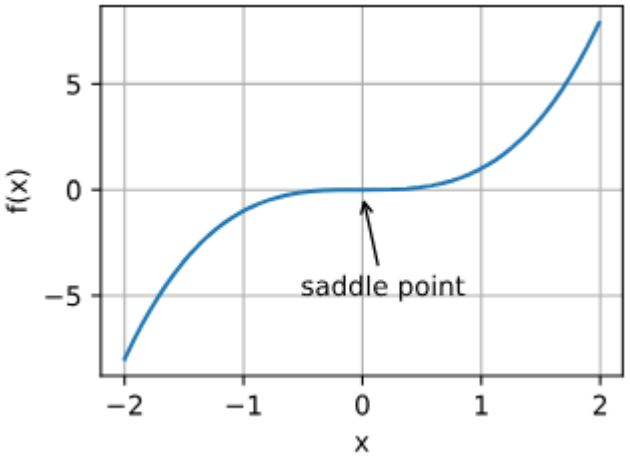
SOLVER	USES GRADIENT	USES CURVATURE	L1 SUPPORT	BEST FOR
Newton	✅	✅ (exact)	❌	Small datasets
LBFGS	✅	✅ (approx)	❌	Default logistic regression
SAGA	✅	❌ (avg grad)	✅	Large data + L1
liblinear	✅	❌	✅	Small data, binary

Challenges of Optimization Algorithm

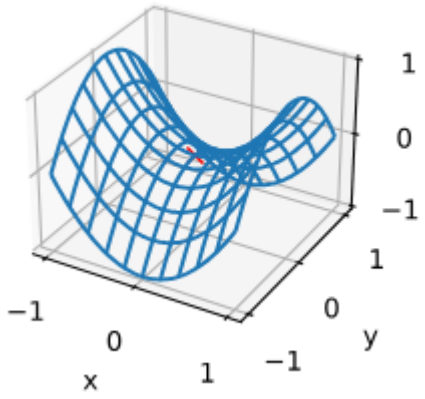
1. **Local Minima**
   - When the numerical solution of an optimization problem is near the local optimum, the numerical solution obtained by the final iteration may only minimize the objective function *locally*, rather than *globally*, as the gradient of the objective function's solutions approaches or becomes zero.
   - The optimization problems may have many local minima.



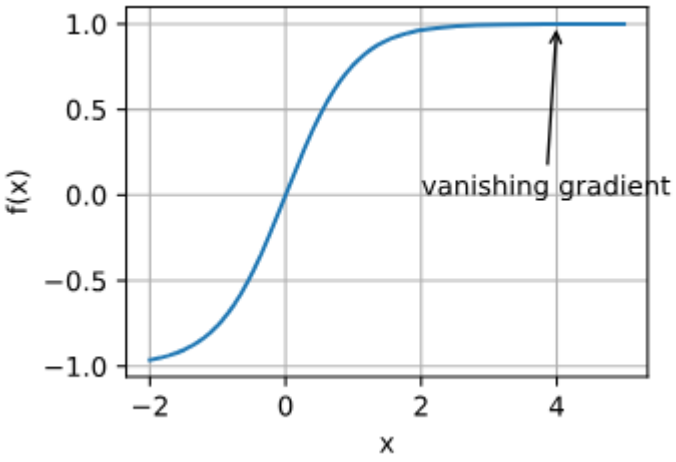
2. **Saddle point**
   - saddle points are another reason for gradients to vanish
   - saddle point is any location where all gradients of a function vanish but which is neither a global nor a local minimum.
   - The problem may have even more saddle points, as generally the problems are not convex.



F(x) = X^3. it's first and second derivative vanish for x = 0. Optimization may stall at this point, even though it is not minimum



3. Vanishing Gradients
   - the most insidious problem to encounter,
   - optimization will get stuck for a long time before we make progress. This turns out to be one of the reasons that training deep learning models was quite tricky prior to the introduction of the ReLU activation function.



- Stochastic Gradient Descent

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{g}(\mathbf{w}_t)$$

- SGD with momentum

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \mathbf{v}_{t+1}$$

$$\mathbf{v}_{t+1} = \rho \mathbf{v}_t - \eta \mathbf{g}(\mathbf{w}_t)$$

Momentum

- AdaGrad = Adaptive gradients

Adaptive Gradient

$$V_t = V_{t-1} + \nabla W_t^2$$

$$W_{t+1} = W_t - \alpha \frac{\nabla W_t}{\sqrt{V_t + \epsilon}}$$

$$\omega_{i,t+1} = \omega_{i,t} - \frac{\eta}{\varepsilon + \sqrt{\sum_{j=1}^{t} g(\omega_{i,j})^2}} g(\omega_{i,t})$$

- RMSProp = Root mean squared Propagation
  - it is built upon an AdaGrad, both optimizer modulate the learning rate for each parameter separately but Adagrad does not weight down the older gradient. it simply adds up the square of all the past gradient.

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + (1-\beta)\mathbf{g}(\mathbf{w}_t)^2$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta}{\varepsilon + \sqrt{\mathbf{v}_{t+1}}} \mathbf{g}(\mathbf{w}_t)$$

- AdaDelta
- Adam = A method for stochastic optimization
  - adaptive learning rate
  - it combines both adaptive gradient and momentum

Adaptive Moment Estimation

$$M_t = \beta_1 M_{t-1} + (1-\beta_1)\nabla W_t$$

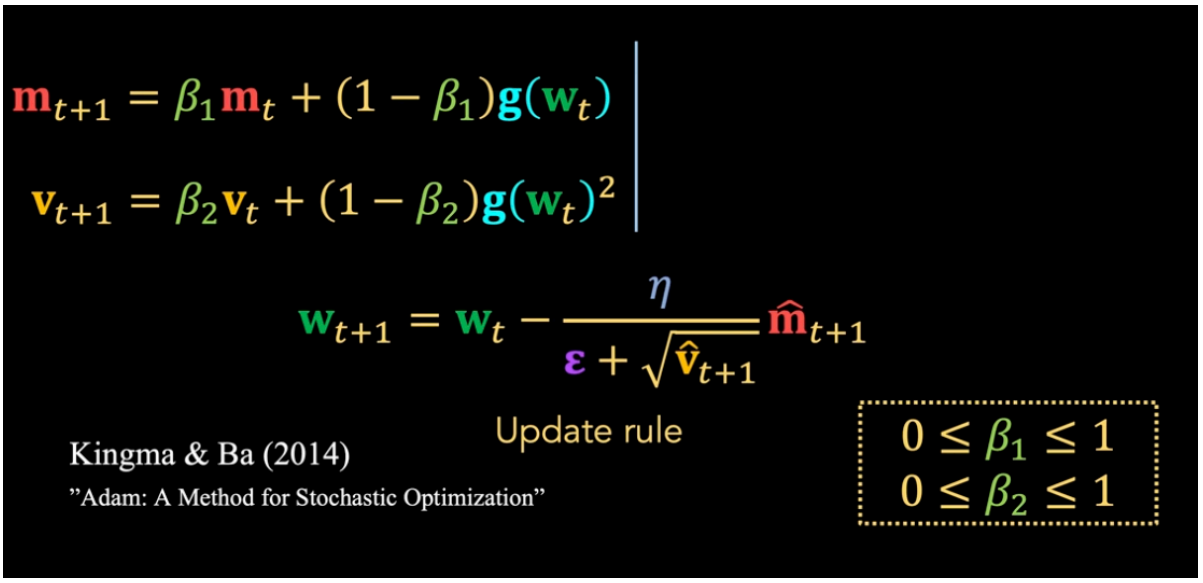
$$V_t = \beta_2 V_{t-1} + (1-\beta_2)\nabla W_t^2$$

$$\hat{M}_t = \frac{M_t}{1-\beta_1^t}$$

$$\hat{V}_t = \frac{V_t}{1-\beta_2^t}$$

$$\beta_1 = 0.9$$
$$\beta_2 = 0.99$$
$$\epsilon = 10^{-8}$$

$$W_{t+1} = W_t - \alpha \frac{\hat{M}_t}{\sqrt{\hat{V}_t} + \epsilon}$$



The RMSProp is better than the Adam when it come to training the reinforcement learning, because when our agent explore new areas of our environment the distribution of training data tend to shift what used to be the best action towards the beginning of the game may not remain the case later on in the game, that mean sample in reinforcement learning are not independent and identically distributed. in such cases, momentum may actually be a detriment to performance due to sample throughout training potentially varying wildly from each other

Adam is energy inefficient